



Fig. 7: HyperTerminal windows for name and icon



Fig. 8: HyperTerminal windows for COM port

chronous mode. To do so, we need to set bits—receive enable (RXEN) and transmit enable (TXEN)—in USART control and status register B (UCSRB), with a 16-bit value of baud rate stored in USART baud rate register UBRRH and UBRRL (UBRR). Refer datasheet for UBRR values corresponding to a baud rate for a particular clock frequency. Frame format is set to 8-bit mode by setting USART register size—UCSZ0 and UCSZ1—in USART control and status register C (UCSRC), which is also the default value when we initialise USART. Stop bit is set as one and parity bit is disabled by default. Mode select (URSEL) bit should be high while writing to UCSRC. This is one of the most commonly used frame formats.

To transmit a data through serial port T_{XDR} , data is written to USART data register (UDR) when transmit buffer is empty, which is indicated by Data Register Empty Flag (UDRE) set in USART



Fig. 9: HyperTerminal windows for selection of baud rate, data bits, parity, stop bits flow control

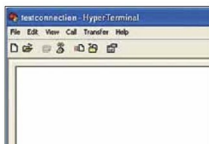


Fig. 10: Sent information on HyperTerminal

control and status register A (UCSRA). Similarly, to receive a data at serial pin R_{XDR} , UDR is read when data is received, which is indicated by receive complete RXC flag set in UCSRA.

USART communication. The step-by-step process is as follows:

1. Initialisation
 - (i) Put UBRR value in UBRR register based on 12-bit baud rate value in UBRRH then UBRRL
 - (ii) Set RXEN and TXEN in UCSRB
 - (iii) Set URSEL, frame format 8-bit by UCSZ1 and UCSZ0
2. USART transmits
 - (i) Wait while Transmit Buffer Empty, UDRE Set in UCSRA
 - (ii) Put data into UDR
 - (iii) USART receives
 - (i) Wait while data to be received, RXC set in UCSRA
 - (ii) Read value in UDR

The steps explained above are demonstrated by asynchronous communication as explained below. The

microcontroller code (USART.C) just adds one to the value that it receives and sends it back to the PC.

Interfacing serial port in Visual Basic.Net. With Windows Vista, HyperTerminal is no longer a part of Windows like Vista. So, we are left with programming environments only to communicate with our devices like microcontrollers. Fig. 6 shows serial communication graphical user interface (GUI) window for handling and monitoring data. This GUI for serial communication is developed using Visual Basic.Net.

The interface demonstrates handling of ASCII text and numeric values through a PC serial port. As per our microcontroller source code (USART.C), it will send back 'B' if you send 'A' and so on. That is, if you enter 'A' under 'Transmit Character' textbox followed by clicking 'Send Character' button, you will see 'B' under 'Feedback' textbox.

Asynchronous serial communication through PC HyperTerminal. Very often we require testing our external embedded devices using serial port. Information is sent to PC through COM port using HyperTerminal program.

To open the HyperTerminal program, go to Start→Programs→Accessories→Communications→HyperTerminal. You will see windows as shown in Fig. 7 for name and icon.

Type in the desired name and click 'OK.' Select COM port (refer Fig. 8) and click 'OK.'

Select the baud rate as '115200', data bit as '8', parity as 'None', and click 'OK' (refer Fig. 9).

Now click 'Transfer' menu bar and send a text file (refer Fig. 10). If the board receives character 'A' it will return 'B' on HyperTerminal. If you press any character, number or symbol then the succeeding character, number or symbol will return on HyperTerminal.

EFY note. The source codes of this article are available on www.efymag.com website.

The author is assistant professor in Department of Mechanical Engineering at Birla Institute of Technology, Mesra